QAPulse

One pulse. From requirement to go-live.

End-to-End Delivery Workflow & Mock-Up Scenario Guide

Purpose	Reference document describing the full QAPulse delivery lifecycle — from milestone creation to closure — with worked scenarios per phase to support mock-up data preparation for presentations and demos.
Audience	PMO, Functional Analysts, Development, QA, and Management stakeholders.
Scope	Milestones, Requirements & AI analysis, Peer review, Development hand-off, Test case preparation, SIT execution, Defect management, UAT, Milestone closure, Risk Register, and Lessons Learned.
Version	1.0 — 17 July 2026

1. Introduction

QAPulse is a delivery management platform that carries a Change Request (CR) through its complete lifecycle on a single system: the PMO plans the milestone, Functional Analysts (FA) author and peer-approve requirements with AI-assisted analysis, the Development team builds against approved requirements, QA prepares and executes test cases, the business signs off through UAT, and the PMO closes the milestone with a retrospective. Two threads run across the entire lifecycle: the **Risk Register**, which is maintained from the day the milestone is created until it is closed, and the **notification engine**, which alerts every involved user at each hand-off.

This document presents the workflow phase by phase. Each phase includes: (a) a short description of who acts and what the system does, and (b) three to four worked scenarios with concrete sample values. The scenarios are intentionally specific so they can be keyed in directly as mock-up data for presentations and demonstrations.

1.1 Roles and Responsibilities

Role	System role key	Responsibility in the flow
PMO / PM Lead	pmo, pm_lead, hod_pm	Creates and plans milestones (dates, test environment ENV1–ENV6), monitors the PM dashboard, moves the milestone through UAT completion, and completes/closes it.
Functional Analyst (FA)	fa_member, fa_lead, hod_fa	Authors requirements and acceptance criteria, runs AI analysis, submits for review. A different FA on the same project approves — an author can never approve their own requirement. FA Lead performs the UAT sign-off.
Dev Lead	dev_lead, hod_dev	Receives notifications when requirements are approved; assigns approved requirements to developers in the team.
Developer	dev_member	Builds the assigned requirement (status In Progress), raises requirement defects when gaps are found, and moves the item to “Ready for QA” when done.
QA Lead	qa_lead, qa_manager, hod_qa	Notified when development starts; plans testing, creates execution files, assigns QA PICs, and monitors pass rates and defects through QA Analytics.
QA Engineer	qa_member	Writes test cases during the development stage, executes SIT and UAT runs, records Pass / Fail / Blocked results, raises defects, and retests fixes.
Admin / CTO	admin, cto	Unrestricted oversight; configures roles, permissions, and audit trail.

1.2 Key Status Values

Object	Status lifecycle
Milestone	planned → active → completed (or cancelled). Completion stamps *completedAt* and *closedBy*.
Requirement — review	draft → in_review → approved / rejected (rejected returns to the author for revision and re-submission).
Requirement — development	assigned → in_progress → ready_for_qa (only possible after the requirement is approved).
Task	new → in_progress → sit / uat → done → released_to_production (blocked when a dependency stalls).
Test execution result	Not Executed → In Progress → Passed / Failed / Blocked.
Defect	New → Assigned/In Progress → Fixed/Resolved → Retest → Closed. Severity: critical / high / medium / low. Found in: SIT / UAT / Production.
Risk	open → mitigating → closed / realized. Severity derived from probability × impact (low / medium / high).

2. Reference Mock-Up Data Set

All scenarios in this document reuse the fictional project, milestone, and people below, so the mock-up data stays consistent from one screen to the next.

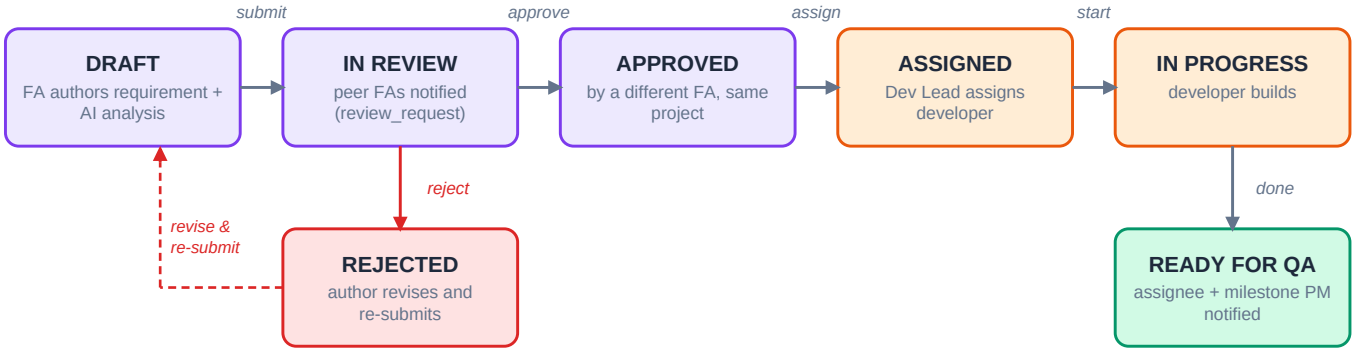
Name	Role	System role	Used in phases
Salmah Idris	PMO	pmo	1, 10, 11, 12
Rizal Hamzah	Head of PM	hod_pm	1, 8, 11
Aina Zulkifli	Functional Analyst (author)	fa_member	2, 3, 5, 9
Daniel Wong	FA Lead (approver / UAT sign-off)	fa_lead	3, 9, 10
Harith Rahman	Functional Analyst (peer reviewer)	fa_member	2, 3
Farhan Abdullah	Dev Lead	dev_lead	4, 5, 7
Wei Jun Tan	Developer	dev_member	4, 5, 7
Kavitha Nair	Developer	dev_member	4, 5
Melissa Lim	QA Lead	qa_lead	5, 6, 7, 8, 9
Syafiq Osman	QA Engineer (QA PIC)	qa_member	6, 7, 8, 9
Nurul Huda	QA Engineer	qa_member	6, 7, 8

Item	Mock value
Project	SPARROW — ePayment Gateway Revamp (status: active)
Milestone	CR-2026-014 “FPX Online Payment Integration” — type: cr, status: planned → active
Key dates	Start 04 Aug 2026 · Requirements target 14 Aug · Dev target 11 Sep · QA target 02 Oct · UAT target 16 Oct · Go-live 23 Oct 2026
Test environment	ENV2 (SIT) — selected and editable by the PM; ENV5 reserved for UAT
Modules	Payment, Wallet, Notification, Reporting
Sample requirements	REQ-101 “FPX single payment (B2C)” · REQ-102 “Payment status inquiry & requery” · REQ-103 “Refund initiation & approval” · REQ-104 “Payment receipt e-mail”

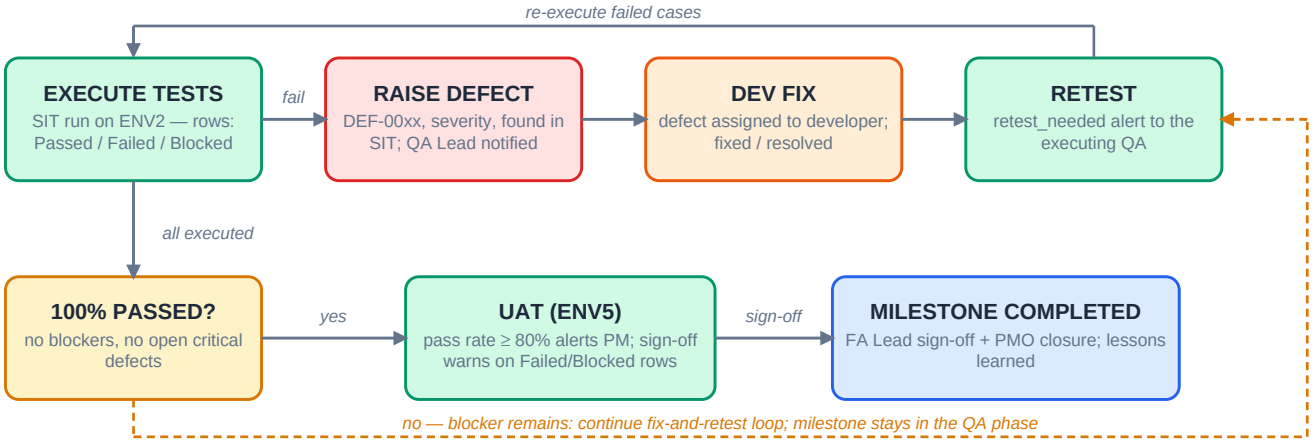
3. End-to-End Workflow Diagram

The diagram below shows the complete lifecycle of a milestone in QAPulse. Boxes are colour-coded by the acting role. Nearly every hand-off fires an in-app notification to the users involved (see the notification matrix in Section 6).

3.1 Requirement Lifecycle (Review and Development Hand-Off)



3.2 Testing, Defect Loop and Exit to UAT



4. Phase-by-Phase Workflow and Scenarios

Each phase below is described with its actors, triggers, and system behaviour, followed by worked scenarios. Scenario identifiers (S1.1, S1.2, ...) can be used as record labels in the mock-up data set.

Phase 1 — Milestone Creation (PMO)

Actor	PMO (Salmah Idris); PM Lead / Head of PM may also create milestones.
Trigger	A Change Request or release is approved for delivery planning.
System behaviour	PMO creates the milestone with type (cr / phase / sprint / release), start date, phase target dates (requirements, development, QA, UAT), planned go-live date, and the test environment (ENV1–ENV6). Status starts as **planned**. The environment remains editable by the PM throughout.
Notifications	Project members see the new milestone on their dashboards; subsequent phase events notify the users involved.
Exit criteria	Milestone exists with dates and environment; FAs can begin attaching requirements.

S1.1 Standard CR milestone with full phase plan

Actors	Salmah Idris (PMO)
Preconditions	Project SPARROW is active; CR-2026-014 has been approved by the steering committee.
Flow	1. Salmah opens Milestones and clicks New Milestone. 2. She enters name “CR-2026-014 — FPX Online Payment Integration”, type cr. 3. She sets start 04 Aug 2026, requirements target 14 Aug, dev target 11 Sep, QA target 02 Oct, UAT target 16 Oct, go-live 23 Oct. 4. She selects test environment ENV2 and saves.
Mock data	status = planned · environment = ENV2 · createdBy = Salmah · requirementCount = 0
Outcome	Milestone appears in the PM dashboard Gantt with baseline dates; FAs on SPARROW can now link requirements to it.

S1.2 Environment changed after creation (PM-editable)

Actors	Salmah Idris (PMO), Melissa Lim (QA Lead)
Preconditions	Milestone CR-2026-014 exists with environment ENV2.
Flow	1. Melissa informs Salmah that ENV2 is booked by another project until mid-September. 2. Salmah edits the milestone and changes the environment from ENV2 to ENV4. 3. The change is recorded in the milestone history.
Mock data	environment: ENV2 → ENV4 · edited 20 Aug 2026 by Salmah
Outcome	QA plans the SIT run against ENV4; the execution files created later reference the updated environment.

S1.3 Urgent hotfix release milestone with a compressed plan

Actors	Rizal Hamzah (Head of PM)
Preconditions	A production issue requires an expedited fix release.
Flow	1. Rizal creates milestone “HOTFIX-2026-003 — Payment Timeout Fix”, type release. 2. Start 07 Sep 2026, dev target 10 Sep, QA target 12 Sep, go-live 13 Sep; UAT phase is waived by agreement. 3. Environment ENV6 is selected because it mirrors production.
Mock data	type = release · environment = ENV6 · duration = 6 days · linked risk R-09 (schedule, high/high)
Outcome	The compressed milestone appears with a visibly short Gantt bar; a schedule risk is logged on day one (see Phase 11).

S1.4 Milestone re-planned before work starts	
Actors	Salmah Idris (PMO)
Preconditions	CR-2026-014 is still in status planned; a dependency (bank sandbox access) slips by two weeks.
Flow	1. Salmah shifts the start date to 18 Aug 2026 and moves every phase target date by two weeks. 2. She records the reason in the milestone notes and raises risk R-02 (external dependency). 3. Status remains planned — no phase has started.
Mock data	startDate: 04 Aug → 18 Aug · goLiveDate: 23 Oct → 06 Nov · risk R-02 external, probability high, impact medium
Outcome	The PM dashboard shows the re-baselined dates; SPI reporting later measures progress against the new plan.

Phase 2 — Requirement Authoring and AI Analysis (FA)

Actor	Functional Analyst (Aina Zulkifli); requirements may also be imported from Redmine as drafts.
Trigger	Milestone exists; FA translates the CR into individual requirements.
System behaviour	FA creates the requirement with title, description, module, priority, release, milestone link and acceptance criteria. The built-in AI analysis reviews the text and returns a quality score (0–100), a risk level (low / medium / high / critical), missing items, and open questions. The analysis is intended for the author and the FAs who review the requirement, and each run is logged to the requirement history. Review status starts at **draft**.
Notifications	The assignee (if set) is notified that a requirement was created or assigned.
Exit criteria	Requirement content is complete enough to submit for peer review.

S2.1 Well-formed requirement passes AI analysis on the first run	
Actors	Aina Zulkifli (FA, author)
Preconditions	Milestone CR-2026-014 exists; Aina is a member of project SPARROW.
Flow	1. Aina creates REQ-101 “FPX single payment (B2C)”, module Payment, priority high, linked to CR-2026-014. 2. She writes five acceptance criteria (e.g. “AC1 — successful payment updates the order to PAID within 5 seconds”). 3. She runs AI analysis.
Mock data	AI score = 88 · riskLevel = low · issues = 0 · missingItems = 1 (“define behaviour when the bank returns a pending status”) · questions = 1
Outcome	Aina adds the pending-status handling to AC6 and the requirement is ready to submit. The AI run is stored in the requirement history.

S2.2 AI flags a weak requirement before review	
Actors	Aina Zulkifli (FA, author)
Preconditions	REQ-103 “Refund initiation & approval” has only a two-line description.
Flow	1. Aina runs AI analysis on the draft. 2. The AI returns a low score with concrete gaps: no maximum refund amount, no approval hierarchy, no handling for partial refunds. 3. Aina rewrites the description and adds seven acceptance criteria. 4. A second AI run confirms the improvement.
Mock data	Run 1: score 46, riskLevel high, missingItems = 4 · Run 2: score 84, riskLevel low, missingItems = 0 · both runs visible in history
Outcome	Rework is prevented before the peer review stage; the score improvement is demonstrable in the requirement history trail.

S2.3 Requirement imported from Redmine and enriched	
Actors	Aina Zulkifli (FA)
Preconditions	Redmine ticket #48213 describes the payment status inquiry feature.
Flow	1. Aina imports #48213; QAPulse creates REQ-102 in status draft with the Redmine ticket linked. 2. Aina completes the module (Payment), priority (medium), milestone link and acceptance criteria. 3. She runs AI analysis before submission.
Mock data	redmineTicketId = 48213 · status = draft · AI score = 79 · riskLevel = medium (“requery interval not specified”)
Outcome	The requirement keeps full traceability back to Redmine while following the QAPulse review workflow.

S2.4 AI duplicate detection prevents overlapping scope	
Actors	Harith Rahman (FA)
Preconditions	REQ-101 (FPX single payment) already exists and is approved.
Flow	1. Harith drafts “REQ-108 — FPX payment for retail buyers”. 2. The AI duplicate check reports 87% similarity with REQ-101. 3. Harith reviews both, confirms the overlap, and instead extends REQ-101’s acceptance criteria via a change comment.
Mock data	duplicate candidate = REQ-101 · similarity = 87% · action = merged, REQ-108 discarded
Outcome	Duplicate scope is caught before development effort is spent; the audit trail records the decision.

Phase 3 — Peer Review and Approval (Segregation of Duties)

Actor	Author submits; a different FA in the same project approves or rejects.
Trigger	Requirement content is complete (review status draft).
System behaviour	Submit moves the requirement to **in_review** and notifies every FA-review-tier user with access to the project (except the submitter). The system enforces segregation of duties: the author cannot approve or reject their own requirement (the action is blocked). Approve stamps approvedBy / approvedAt; reject stamps rejectedBy / rejectedAt and returns the item to the author. Items pending more than three days are flagged as stale in the review queue.
Notifications	review_request (to eligible reviewers) · review_approved (author + assignee) · review_rejected (author, assignee, milestone PM).
Exit criteria	Requirement is **approved** — the precondition for any development assignment.

S3.1 Straight-through approval by a peer FA	
Actors	Aina (author), Harith (peer FA reviewer)
Preconditions	REQ-101 is complete; both FAs are members of project SPARROW.
Flow	1. Aina submits REQ-101 for review (status in_review). 2. Harith and Daniel receive a review_request notification; Aina does not receive one. 3. Harith opens the requirement, reads the AI analysis in the history, and approves.
Mock data	reviewStatus: draft → in_review → approved · approvedBy = Harith · approvedAt = 12 Aug 2026 10:42
Outcome	Aina receives review_approved; REQ-101 becomes eligible for the Dev Lead’s assignment queue (Phase 4).

S3.2 Author attempts to approve their own requirement (blocked)	
Actors	Aina (author)
Preconditions	REQ-102 is in_review and was authored by Aina.
Flow	1. Aina opens REQ-102 and attempts the Approve action. 2. The system rejects the action: “Authors cannot approve or reject their own requirement.” 3. The requirement remains in_review until another FA acts.
Mock data	HTTP 403 on approve · reviewStatus unchanged (in_review) · audit log records the blocked attempt
Outcome	Segregation of duties is demonstrably enforced — a key governance talking point for the presentation.

S3.3 Rejection, revision and re-approval cycle	
Actors	Aina (author), Daniel Wong (FA Lead, reviewer)
Preconditions	REQ-103 (refunds) is in_review.
Flow	1. Daniel rejects REQ-103 with the comment “Approval hierarchy conflicts with Finance SOP — refunds above RM 5,000 need two approvers.” 2. Aina receives review_rejected; the milestone PM is also notified. 3. Aina updates AC4 and re-submits. 4. Daniel approves the revised version.
Mock data	Cycle 1: rejected 13 Aug (rejectedBy = Daniel) · Cycle 2: approved 15 Aug · first-pass metric on the PM dashboard reflects one rejection
Outcome	The rework cycle is visible in the phase breakdown analytics; the requirement proceeds with a corrected rule.

S3.4 Stale review escalated after three days	
Actors	Harith (author), Daniel Wong (FA Lead)
Preconditions	REQ-104 (receipt e-mail) was submitted on 10 Aug and no reviewer has acted by 14 Aug.
Flow	1. The review queue flags REQ-104 as stale (> 3 days). 2. Daniel, seeing the team-wide queue as FA Lead, picks it up. 3. He approves after a minor comment on the e-mail template reference.
Mock data	submitted 10 Aug · stale flag on 14 Aug · approved 14 Aug 16:05 by Daniel
Outcome	The stale indicator prevents requirements from silently ageing in review and keeps the requirements phase on schedule.

Phase 4 — Development Assignment (Dev Lead)

Actor	Dev Lead (Farhan Abdullah) — assignment is restricted to lead-tier roles.
Trigger	A requirement reaches **approved**; it appears in the dev queue as Unassigned.
System behaviour	The Dev Lead assigns an approved requirement to a developer (dev status **assigned**). The system refuses to assign a requirement that is not yet approved. The assigned developer is notified (requirement_dev_assigned). Developers see their own items under “My Dev Work”.
Notifications	requirement_dev_assigned (developer) · task assignment notifications where tasks are used.
Exit criteria	Every approved requirement in the milestone has a named developer.

S4.1 Dev Lead assigns an approved requirement	
Actors	Farhan (Dev Lead), Wei Jun (Developer)
Preconditions	REQ-101 is approved (S3.1) and shows in the Unassigned dev queue.
Flow	1. Farhan opens the dev queue and sees REQ-101 under Unassigned. 2. He assigns Wei Jun, who owns the Payment module integrations. 3. Wei Jun receives the requirement_dev_assigned notification.
Mock data	devStatus = assigned · devAssigneeId = Wei Jun · devAssignedBy = Farhan · devAssignedAt = 17 Aug 2026 09:15
Outcome	REQ-101 appears in Wei Jun's "My Dev Work" list, ready to start.

S4.2 Assignment of an unapproved requirement is refused	
Actors	Farhan (Dev Lead)
Preconditions	REQ-103 is still in its rejection/revision cycle (S3.3), i.e. not approved.
Flow	1. Farhan attempts to assign REQ-103 to Kavitha to get a head start. 2. The system blocks the action: a requirement must be FA-approved before development assignment. 3. Farhan waits for the approval on 15 Aug and assigns it the same day.
Mock data	HTTP 409 "requirement not approved" · assignment succeeds only after reviewStatus = approved
Outcome	The gate guarantees developers only ever build against approved, reviewed scope.

S4.3 Mid-flight reassignment due to planned leave	
Actors	Farhan (Dev Lead), Kavitha and Wei Jun (Developers)
Preconditions	REQ-102 was assigned to Kavitha and is in progress; Kavitha starts two weeks of leave on 01 Sep.
Flow	1. Farhan reassigns REQ-102 from Kavitha to Wei Jun on 29 Aug. 2. Wei Jun is notified; the requirement history shows both assignments. 3. Kavitha's handover notes are attached as a comment.
Mock data	devAssigneeId: Kavitha → Wei Jun · reassigned 29 Aug · handover comment linked
Outcome	Continuity is preserved and the resource change is fully auditable.

S4.4 Batch assignment across the team by module	
Actors	Farhan (Dev Lead)
Preconditions	REQ-101, REQ-102 and REQ-104 are all approved on the same day.
Flow	1. Farhan reviews the three approved requirements in the queue. 2. He assigns Payment-module items (REQ-101, REQ-102) to Wei Jun and the Notification-module item (REQ-104) to Kavitha. 3. Both developers receive one notification per assignment.
Mock data	3 assignments · Wei Jun = 2 requirements · Kavitha = 1 requirement · all devStatus = assigned
Outcome	Workload is spread by module ownership; the Resources view reflects each developer's active load.

Phase 5 — Development Build and Parallel QA Planning

Actor	Developers build; QA Lead plans testing in parallel.
Trigger	Developer starts an assigned requirement (dev status **in_progress**).

System behaviour	While development runs, the QA Lead is aware of the incoming scope (approved requirements and dev progress are visible on the milestone) and prepares the test plan: execution files are created for the milestone, QA PICs are assigned, and the environment booking is confirmed. Developers who find gaps can raise a requirement defect back to the FA. Task dependencies can be declared (blocked-by) and completion is tracked in hours and percent.
Notifications	execution (QA PIC assignment) · defect_opened (requirement defect → author) · task status notifications to assignees.
Exit criteria	Developer marks the requirement **ready_for_qa**; assignee and milestone PM are notified.

S5.1 Build starts; QA Lead stands up the test plan	
Actors	Wei Jun (Developer), Melissa Lim (QA Lead), Syafiq (QA)
Preconditions	REQ-101 assigned to Wei Jun (S4.1).
Flow	1. Wei Jun sets REQ-101 to in_progress on 18 Aug. 2. Melissa creates execution file “SIT — CR-2026-014 Payment” (fileType qa) linked to the milestone and REQ-101. 3. She assigns Syafiq as QA PIC; Syafiq receives the execution notification. 4. ENV4 booking is confirmed for the SIT window.
Mock data	devStatus = in_progress · executionFile: title “SIT — CR-2026-014 Payment”, qaPic = Syafiq, modules = Payment, Wallet
Outcome	Test planning starts on day one of the build instead of waiting for hand-over — the core “parallel QA” message for the demo.

S5.2 Developer raises a requirement defect during the build	
Actors	Wei Jun (Developer), Aina (FA author)
Preconditions	REQ-101 in progress; the FPX specification is ambiguous about duplicate payment references.
Flow	1. Wei Jun raises a requirement defect on REQ-101: “AC3 does not define behaviour when the bank returns a duplicate transaction reference.” 2. Aina is notified (defect_opened) as the requirement author. 3. Aina clarifies AC3; because the description changed, linked test cases and tasks are flagged for re-review (revision_required).
Mock data	DEF-0031 · severity = medium · type = requirement defect · revision_required sent to Syafiq (linked TC author)
Outcome	The gap is resolved in the requirement — not silently patched in code — and QA knows their draft test cases need a review.

S5.3 Task blocked by an upstream dependency	
Actors	Kavitha (Developer), Farhan (Dev Lead)
Preconditions	Task T-88 “Receipt e-mail template” (REQ-104) depends on task T-85 “SMTP relay configuration”.
Flow	1. Kavitha marks T-88 as blocked with blocked-by T-85. 2. The dependency shows on the milestone task board; T-88 cannot be closed while T-85 is open. 3. When T-85 completes on 05 Sep, Kavitha resumes and moves T-88 to in_progress.
Mock data	T-88.status = blocked · blockedByTaskId = T-85 · blocked for 4 working days
Outcome	The blocker is visible to the PM dashboard’s top-blockers panel instead of hiding inside a developer’s inbox.

S5.4 Build completed — hand-over to QA	
Actors	Wei Jun (Developer), Salmah (PMO), Syafiq (QA)
Preconditions	REQ-101 build and unit tests are done; code is deployed to ENV4.
Flow	1. Wei Jun sets REQ-101 to “Ready for QA” on 09 Sep. 2. The requirement assignee and the milestone PM (Salmah) receive requirement_ready_for_qa notifications. 3. Syafiq schedules the SIT run to begin 10 Sep.
Mock data	devStatus = ready_for_qa · readyForQaAt = 09 Sep 2026 17:30 · notification “Ready for QA — REQ-101”
Outcome	A clean, time-stamped hand-over from development to QA, two days ahead of the dev target date.

Phase 6 — Test Case Preparation During Development (QA)

Actor	QA Engineers (Syafiq, Nurul), guided by the QA Lead.
Trigger	Requirements are approved and development has started — QA does not wait for the build to finish.
System behaviour	QA writes test cases in the library: title, objective, preconditions, test steps, test data, expected result, tags, and the linked requirement. AI assistance can generate draft test cases and edge cases (marked aiAssisted). If a requirement description changes after test cases exist, the affected test cases are flagged for review and their authors notified. Coverage tooling shows requirements without any linked test case.
Notifications	revision_required (when a linked requirement is revised).
Exit criteria	Every acceptance criterion of every approved requirement is covered by at least one test case.

S6.1 Manual test cases authored from acceptance criteria	
Actors	Syafiq (QA)
Preconditions	REQ-101 approved with six acceptance criteria; build in progress.
Flow	1. Syafiq writes TC-201 “Successful FPX payment updates order to PAID”: preconditions (registered buyer, active FPX bank), 7 steps, test data (order MYR 150.00, bank = Maybank2u sandbox), expected result. 2. He repeats for AC2–AC6, producing TC-202 to TC-206. 3. Each test case links back to REQ-101.
Mock data	6 test cases · authorId = Syafiq · status = active · tags = fpx, payment, sit
Outcome	Full acceptance-criteria coverage exists before the code arrives, enabling SIT to start the day after hand-over.

S6.2 AI-assisted edge case generation	
Actors	Nurul (QA)
Preconditions	TC-201 to TC-206 cover the happy paths of REQ-101.
Flow	1. Nurul runs the AI edge-case generator on REQ-101. 2. The AI proposes: payment abandoned at the bank page, session timeout after debit, duplicate browser tab double-submit, and amount mismatch on callback. 3. Nurul accepts three proposals and edits one, creating TC-207 to TC-210 marked aiAssisted.
Mock data	4 new TCs · aiAssisted = true · e.g. TC-209 “Callback amount mismatch is rejected and flagged for reconciliation”
Outcome	Negative-path coverage improves measurably; the aiAssisted flag lets the demo show AI contribution statistics.

S6.3 Requirement revision invalidates existing test cases	
Actors	Aina (FA), Syafiq (QA)
Preconditions	REQ-101 description was clarified after the requirement defect in S5.2; TC-201–TC-210 already exist.
Flow	1. Aina's description change stamps the linked test cases as requiring review. 2. Syafiq receives revision_required. 3. He updates TC-203 (duplicate reference handling) to match the clarified AC3 and acknowledges the revision flag on the rest.
Mock data	requirementRevisedAt = 26 Aug · 1 TC updated, 9 acknowledged unchanged
Outcome	Test assets never silently drift out of date with the requirement they verify.

S6.4 Coverage gap detected and closed	
Actors	Melissa (QA Lead), Nurul (QA)
Preconditions	REQ-104 (receipt e-mail) was approved late (S3.4) and has no test cases yet.
Flow	1. Melissa runs the coverage check for CR-2026-014: REQ-104 shows zero linked test cases. 2. She assigns Nurul, who writes TC-215 to TC-217 (successful e-mail, invalid address bounce, template rendering in BM and English). 3. Coverage returns to 100% of approved requirements.
Mock data	coverage before = 3/4 requirements (75%) · after = 4/4 (100%) · 3 new TCs
Outcome	The coverage snapshot on QA Analytics is the objective gate for "ready to test".

Phase 7 — QA Test Execution — SIT

Actor	QA PIC (Syafiq) and QA team execute; developers fix; QA Lead monitors.
Trigger	Developer set the requirement to **ready_for_qa** (S5.4).
System behaviour	QA executes the SIT execution file row by row on the milestone's test environment. Each row records result (Passed / Failed / Blocked / In Progress), actual result, executed time, defect number and screenshots. Failures raise defects (severity, module, found in SIT) which notify the QA Lead tier and are assigned to developers. When a defect is marked fixed while its linked test case is still Failed, the executing QA receives a retest_needed alert. Module summaries and pass rates recalculate on every save and stream to the dashboards.
Notifications	defect_opened (QA Lead+) · defect_assigned (developer) · defect_status_changed (reporter + executor) · retest_needed (executor).
Exit criteria	All rows executed; every defect either closed or explicitly dispositioned.

S7.1 Clean first execution round	
Actors	Syafiq (QA PIC)
Preconditions	SIT file contains 20 rows (TC-201–TC-210 plus regression rows); build deployed to ENV4.
Flow	1. Syafiq executes rows 1–20 across 10–12 Sep. 2. 18 rows pass on the first attempt; 2 rows remain In Progress pending bank sandbox data. 3. On 13 Sep the remaining 2 rows pass.
Mock data	results: 20 Passed / 0 Failed / 0 Blocked · pass rate 100% · module summary: Payment 14, Wallet 6
Outcome	Best-case demo data: the milestone's QA phase completes with zero defects and the pass-rate widget shows 100%.

S7.2 Failure, defect, fix and retest — the full defect loop	
Actors	Nurul (QA), Melissa (QA Lead), Wei Jun (Developer)
Preconditions	TC-209 “Callback amount mismatch” is being executed.
Flow	1. TC-209 fails: the mismatched callback is accepted instead of rejected. 2. Nurul raises DEF-0042 (severity high, module Payment, found in SIT) with steps to reproduce and a screenshot; Melissa’s QA-Lead tier is notified. 3. Melissa assigns DEF-0042 to Wei Jun (defect_assigned). 4. Wei Jun fixes and marks it Resolved; because TC-209 is still Failed, Nurul receives retest_needed. 5. Nurul retests on 16 Sep — Passed — and DEF-0042 is Closed.
Mock data	DEF-0042 · severity = high · foundIn = SIT · open 14 Sep → closed 16 Sep (2 days) · linked to TC-209 and REQ-101
Outcome	One complete defect lifecycle with every notification type — the richest single scenario for the mock-up.

S7.3 Environment outage blocks execution	
Actors	Syafiq (QA PIC), Melissa (QA Lead), Salmah (PMO)
Preconditions	ENV4 database is taken down by an infrastructure patch mid-run.
Flow	1. Syafiq marks the 6 remaining rows as Blocked with the comment “ENV4 outage — infra ticket INC-7731”. 2. Melissa raises risk R-05 (technical, probability medium, impact high) on the register. 3. ENV4 is restored after 1.5 days; rows are re-executed and pass. 4. Salmah notes the schedule impact on the milestone.
Mock data	6 rows Blocked for 1.5 days · risk R-05 status open → mitigating → closed · QA target still met
Outcome	Shows how Blocked results, the risk register and PM visibility interlock during an incident.

S7.4 AI regression selection before the final round	
Actors	Melissa (QA Lead)
Preconditions	The fix for DEF-0042 touched shared payment-callback code.
Flow	1. Melissa runs AI regression selection scoped to the changed area. 2. The AI recommends 8 regression test cases from the library (wallet top-up callback, refund callback, status requery). 3. The 8 rows are appended to the SIT file and executed — all pass.
Mock data	8 regression rows added · execution 17 Sep · all Passed · aiAssisted selection logged
Outcome	Targeted regression instead of a blanket re-run — a strong efficiency story for the presentation.

Phase 8 — End of Testing — 100% or Blocker (Milestone Progress Update)

Actor	QA Lead consolidates results; QA members update their milestone-linked tasks; PM reviews.
Trigger	All planned SIT rows have been executed at least once.
System behaviour	The decision gate: proceed to UAT only when execution is 100% complete with no blocking defects. Module summaries (passed / failed / blocked / not executed) and the milestone pass rate feed the PM dashboard, whose work-completed percentage and Schedule Performance Index (SPI) are derived from the QA roll-up. QA members update task status and completion percentage against the milestone. Open critical defects appear as top blockers.
Notifications	task update notifications to assignees; dashboard broadcasts on every execution save.
Exit criteria	100% executed, pass rate at the agreed threshold, zero open blocker/critical defects — or a documented deferral decision.

S8.1 Clean exit — 100% passed, gate opens to UAT	
Actors	Melissa (QA Lead), Salmah (PMO)
Preconditions	All 28 SIT rows (20 planned + 8 regression) executed.
Flow	1. Final tally: 28 Passed / 0 Failed / 0 Blocked; DEF-0042 closed. 2. Melissa updates the milestone QA tasks to done with 100% completion. 3. Salmah sees work-completed reach the QA-phase target on the dashboard and green-lights UAT preparation.
Mock data	pass rate = 100% · open defects = 0 · SPI = 1.04 (slightly ahead of schedule)
Outcome	The milestone crosses the QA gate on objective numbers, not opinion.

S8.2 96% pass with one deferred medium defect	
Actors	Melissa (QA Lead), Daniel (FA Lead), Salmah (PMO)
Preconditions	27 of 28 rows passed; DEF-0047 (cosmetic receipt alignment, severity medium) remains open.
Flow	1. Melissa presents the result: 96.4% pass, one open medium defect with no functional impact. 2. Daniel and Salmah agree to defer DEF-0047 to the next release; the decision is recorded on the defect and the milestone notes. 3. The gate to UAT opens with the deferral documented.
Mock data	pass rate = 96.4% · DEF-0047 status = Deferred · deferral decision note dated 19 Sep
Outcome	Demonstrates governed flexibility: not every defect blocks UAT, but every deferral is explicit and auditable.

S8.3 Blocker halts the milestone in the QA phase	
Actors	Nurul (QA), Melissa (QA Lead), Farhan (Dev Lead), Rizal (Head of PM)
Preconditions	DEF-0051 “Refund double-posts on retry” (severity critical) found on 20 Sep.
Flow	1. DEF-0051 is raised; the pass rate drops and the defect appears in the PM dashboard top blockers. 2. UAT preparation is suspended; the milestone stays in the QA phase and SPI falls below 1.0. 3. Farhan pulls Wei Jun and Kavitha onto the fix; it lands 24 Sep. 4. Retest plus the AI-selected regression pack pass on 25 Sep and the gate re-opens.
Mock data	DEF-0051 severity = critical · SPI dip to 0.91 · QA phase extended 4 days · risk R-06 (scope/technical) logged
Outcome	The “blocker branch” of the decision diamond in Section 3.2, played out end to end.

S8.4 QA task roll-up keeps the milestone percentage honest	
Actors	Syafiq and Nurul (QA)
Preconditions	Milestone tasks exist for SIT execution, regression and UAT support.
Flow	1. Syafiq updates task “SIT execution — Payment” to done, actual hours 38 vs estimate 40. 2. Nurul updates “Regression round” to done, 12 hours. 3. “UAT support” moves to in_progress at 25% completion. 4. Task assignees receive the status-change notifications.
Mock data	3 tasks · completion: 100% / 100% / 25% · actualHours captured per task
Outcome	The milestone progress figure the PMO reports upward is reconstructed from task-level facts.

Phase 9 — User Acceptance Testing (UAT)

Actor	Business users execute; FA facilitates; QA supports; PM monitors.
Trigger	SIT gate passed (Phase 8); UAT execution file created (fileType uat) on the UAT environment.

System behaviour	UAT runs as a dedicated execution file separate from SIT, so UAT progress and pass rates are tracked independently. When the UAT pass rate crosses 80%, the milestone PM automatically receives a uat_milestone_ready notification (fired once per milestone). Defects found here are recorded with foundIn = UAT, feeding the defect escape funnel (SIT vs UAT vs Production) on QA Analytics.
Notifications	uat_milestone_ready (milestone PM, at ≥ 80% pass) · defect notifications as in SIT.
Exit criteria	UAT scripts pass and the business is ready to sign off.

S9.1 UAT kick-off and the 80% readiness alert

Actors	Aina (FA facilitator), business users, Salmah (PMO)
Preconditions	UAT file “UAT — CR-2026-014” created with 15 business scripts; ENV5 prepared with masked production-like data.
Flow	1. Business users execute scripts across 06–09 Oct. 2. By 08 Oct, 13 of 15 scripts have passed (86.7%). 3. Crossing 80% fires uat_milestone_ready to Salmah — once only. 4. The remaining 2 scripts pass on 09 Oct.
Mock data	fileType = uat · 15 rows · pass rate 86.7% → 100% · notification timestamp 08 Oct 14:20
Outcome	The PM learns UAT is nearly done without chasing anyone — a favourite demo moment.

S9.2 Defect found in UAT feeds the escape funnel

Actors	Business user, Syafiq (QA support), Wei Jun (Developer)
Preconditions	UAT script 11 covers refunds initiated from the back office.
Flow	1. A business user finds that the refund confirmation shows the gross amount instead of the net amount. 2. Syafiq logs DEF-0058, severity medium, foundIn = UAT. 3. The escape funnel on QA Analytics now shows 1 UAT escape against 3 SIT catches for this milestone. 4. Wei Jun fixes; the script is re-run and passes on 13 Oct.
Mock data	DEF-0058 · foundIn = UAT · escape funnel: SIT 3 / UAT 1 / Production 0
Outcome	Escape metrics give QA a concrete improvement conversation: why did SIT miss it? (Answer feeds Lessons Learned.)

S9.3 Sign-off attempted with an outstanding failed UAT row

Actors	Daniel Wong (FA Lead)
Preconditions	Script 11 is still marked Failed while DEF-0058 is being fixed.
Flow	1. Daniel opens the milestone sign-off on 12 Oct. 2. The system warns: outstanding UAT rows with result Failed/Blocked exist. 3. Daniel holds the sign-off, waits for the 13 Oct retest, and signs off on 14 Oct with a clean sheet.
Mock data	sign-off warning logged 12 Oct · final sign-off 14 Oct · UAT pass rate 100%
Outcome	The system nudges — but does not silently block — governance at sign-off, and the caution is auditable.

S9.4 UAT completes at 100% within the target date	
Actors	Business users, Aina (FA), Salmah (PMO)
Preconditions	All 15 scripts pass, DEF-0058 closed.
Flow	1. Final UAT summary: 15/15 Passed, zero open defects. 2. Aina compiles the UAT evidence pack from the execution file (results, timestamps, screenshots). 3. Salmah confirms UAT completion two days ahead of the 16 Oct target.
Mock data	UAT pass rate = 100% · uatFileCount = 1 · completed 14 Oct vs target 16 Oct
Outcome	The milestone is ready for formal closure with a complete, system-generated evidence trail.

Phase 10 — UAT Completion and Milestone Closure (PMO)

Actor	FA Lead signs off; PMO completes and closes the milestone.
Trigger	UAT passed and evidence compiled (Phase 9).
System behaviour	The FA Lead's sign-off approval moves the milestone to **completed**; the system stamps completedAt and records closedBy. A rejected sign-off returns the milestone to planned for rework. The PMO confirms go-live, distributes the PMO report (pass rates per module and ticket, e-mailed as HTML/PDF), and closes out remaining tasks as released_to_production.
Notifications	Sign-off outcome visible to the project; PMO report e-mailed to stakeholders.
Exit criteria	Milestone completed and closed; production release confirmed.

S10.1 Sign-off, completion and go-live	
Actors	Daniel (FA Lead), Salmah (PMO)
Preconditions	UAT 100% (S9.4).
Flow	1. Daniel approves the milestone sign-off on 14 Oct; status becomes completed, completedAt is stamped, closedBy recorded. 2. Salmah confirms go-live for 23 Oct as planned. 3. The PMO report for CR-2026-014 is generated and e-mailed to stakeholders with per-module pass rates.
Mock data	status = completed · completedAt = 14 Oct 2026 15:02 · closedBy = Daniel · go-live 23 Oct
Outcome	The milestone closes on objective evidence, with an automatically generated stakeholder report.

S10.2 Sign-off rejected — milestone returned for rework	
Actors	Daniel (FA Lead), Melissa (QA Lead)
Preconditions	Alternative timeline: the business reports an unresolved usability concern on the refund screen at sign-off.
Flow	1. Daniel rejects the sign-off with reasons; the milestone returns to planned. 2. The concern is logged as DEF-0060 (severity medium) and scheduled. 3. After the fix and a focused UAT re-run, Daniel approves on the second attempt.
Mock data	sign-off action = reject → status planned · second sign-off approved 20 Oct · closure slips 6 days
Outcome	Shows the rework branch of closure: rejection is a controlled loop, not an exception handled outside the system.

S10.3 Post-completion housekeeping — tasks released to production	
Actors	Salmah (PMO), Farhan (Dev Lead)
Preconditions	Milestone completed (S10.1); go-live executed on 23 Oct.
Flow	1. After go-live verification, Farhan moves the delivery tasks to released_to_production. 2. Salmah checks the Resources view: the SPARROW team's active milestone count drops, freeing capacity. 3. The milestone is archived from active dashboards but remains fully auditable.
Mock data	5 tasks → released_to_production · milestone visible under closed milestones in Resources
Outcome	A tidy end state: nothing left dangling in "in progress" after the release.

S10.4 Production escape after closure triggers an escape review	
Actors	Support intake, Melissa (QA Lead), Nurul (QA)
Preconditions	Two weeks after go-live, a customer reports that receipts for zero-decimal amounts show "RM 100." instead of "RM 100.00".
Flow	1. DEF-P0004 is logged with source = production, foundIn = Production, linked to the closed milestone. 2. The escape review classifies it: coverage gap (no test data with whole-ringgit amounts). 3. TC-224 is added to the regression library so the gap never recurs.
Mock data	DEF-P0004 · escapeClass = coverage_gap · escape funnel: SIT 3 / UAT 1 / Production 1 · new regression TC added
Outcome	Even post-closure defects loop back into the quality system — the closing argument for continuous improvement.

Phase 11 — Risk Register (Continuous — Start to End of the Milestone)

Actor	Anyone lead-tier and above raises risks; owners drive mitigation; PM reviews on the dashboard.
Trigger	Continuous — from the day the milestone is created until it is closed.
System behaviour	Each risk records title, description, category (schedule / scope / resource / technical / external / other), probability and impact (low / medium / high), an owner, a mitigation plan and a status (open → mitigating → closed / realized). Severity is derived from probability × impact. The register is embedded in the PM dashboard, and the AI milestone risk assessment can synthesise an overall risk level with contributing factors from live delivery data.
Notifications	Risks surface on the PM dashboard and in the AI risk assessment history.
Exit criteria	At closure, every risk is closed or realized (with consequences recorded).

S11.1 Schedule risk raised on day one	
Actors	Salmah (PMO, raiser and owner)
Preconditions	Milestone created (S1.1); the bank's FPX sandbox onboarding normally takes three weeks.
Flow	1. Salmah raises R-01 "Bank sandbox credentials may arrive after the dev start date" — category external, probability high, impact medium. 2. Mitigation: submit onboarding forms immediately; develop against the FPX simulator until credentials arrive. 3. Credentials arrive 22 Aug; R-01 moves to closed.
Mock data	R-01 · external · P=high, I=medium (severity medium-high) · status open → mitigating → closed 22 Aug
Outcome	The register starts alive on day one, not as an end-of-project formality.

S11.2 Technical risk during development moves to mitigating	
Actors	Farhan (Dev Lead, raiser), Wei Jun (owner)
Preconditions	Callback signature verification depends on a library with a known deprecation notice.
Flow	1. Farhan raises R-04 “Crypto library deprecation may force a late swap” — technical, probability medium, impact high. 2. Mitigation: isolate the verification behind an interface so the library can be replaced without touching business logic. 3. Status set to mitigating while the abstraction is built; closed after SIT passes.
Mock data	R-04 · technical · P=medium, I=high · mitigationPlan documented · closed 25 Sep
Outcome	Architecture decisions are traceably linked to a named risk.

S11.3 Resource risk realized and absorbed	
Actors	Melissa (QA Lead, raiser and owner)
Preconditions	Nurul is called for two days of jury-style committee duty in the middle of the SIT window.
Flow	1. Melissa raises R-07 “QA capacity dip during SIT week 2” — resource, probability high, impact medium. 2. Mitigation: Syafiq absorbs the priority rows; non-critical regression shifts by one day. 3. The absence happens as feared — R-07 is marked realized, with the actual impact (1 day slip, recovered) recorded.
Mock data	R-07 · resource · status = realized · actual impact note: “1 day slip on regression, recovered within the QA window”
Outcome	The register honestly distinguishes risks that were avoided from risks that happened — credibility for the PMO’s reporting.

S11.4 AI milestone risk assessment for the steering meeting	
Actors	Rizal (Head of PM)
Preconditions	Mid-September: DEF-0051 (critical) is open and SPI has dipped to 0.91 (S8.3).
Flow	1. Rizal runs the AI milestone risk assessment before the steering meeting. 2. The AI aggregates phase timelines, review KPIs, the risk register, open defects and coverage, and returns riskLevel = high with three factors (critical defect open, SPI below 1.0, UAT window compression) and a suggested mitigation. 3. The assessment is stored append-only in the history for comparison next week.
Mock data	riskLevel = high · factors = 3 · next assessment (28 Sep) returns medium after the fix lands
Outcome	Management gets a defensible, data-derived risk narrative instead of a gut-feel RAG status.

Phase 12 — Lessons Learned (After Milestone Closure)

Actor	PMO facilitates; all roles contribute.
Trigger	Milestone reaches **completed**; the retrospective is recorded on the milestone itself (lessons learned, closed-by, completion timestamp).
System behaviour	Lessons are captured against the closed milestone so they stay attached to the delivery they came from. Inputs come from the phase analytics: rework cycles, first-pass approval rate, defect escape funnel, blocker history and risk outcomes.
Notifications	Retrospective summary circulated with the final PMO report.
Exit criteria	Lessons recorded; actionable items converted into process changes or backlog entries for the next milestone.

S12.1 What went well — parallel QA and AI analysis cut the timeline	
Actors	Salmah (PMO), all leads
Preconditions	CR-2026-014 closed (S10.1).
Flow	1. In the retrospective, the team reviews the phase breakdown: SIT started one day after dev hand-over because test cases were written during the build (S6.1). 2. AI analysis prevented one rework cycle on REQ-103 (S2.2). 3. Both practices are recorded as “continue” items in the milestone’s lessons learned.
Mock data	lessonsLearned entry 1: “Test case authoring in parallel with development saved ~5 working days; make it the default for all CRs.”
Outcome	Positive practices are institutionalised with numbers attached.

S12.2 What to fix — environment booking clashes	
Actors	Melissa (QA Lead), Salmah (PMO)
Preconditions	The ENV2 → ENV4 switch (S1.2) and the ENV4 outage (S7.3) both cost time.
Flow	1. The retrospective traces 2.5 lost days to environment contention and an unannounced infra patch. 2. Lesson recorded: introduce a shared environment booking calendar and a change-freeze notice for active SIT windows. 3. An action item is assigned to the PMO to pilot the calendar next quarter.
Mock data	lessonsLearned entry 2: “ENV contention cost 2.5 days; adopt a booking calendar + freeze windows.” · action owner = PMO
Outcome	A concrete, owned process improvement — not a vague observation.

S12.3 What escaped — SIT test data gap behind the UAT and production defects	
Actors	Melissa (QA Lead), Syafiq and Nurul (QA)
Preconditions	Escape funnel for the milestone: SIT 3 / UAT 1 / Production 1 (S9.2, S10.4).
Flow	1. The team performs a mini root-cause review on DEF-0058 (UAT) and DEF-P0004 (production). 2. Both trace to test-data variety: no back-office refund path data and no whole-ringgit amounts in SIT. 3. Lesson recorded: expand the standard test-data pack; two regression TCs already added (S10.4) are cited as the fix.
Mock data	lessonsLearned entry 3: “Both escapes were test-data gaps, not procedure gaps; standard data pack extended (TC-224, TC-225).”
Outcome	The escape funnel converts directly into permanent regression assets — closing the quality loop that the whole document describes.

5. Notification Matrix

Almost every hand-off in the workflow notifies the users involved. The matrix below lists the main notification events in lifecycle order, and is useful for scripting the inbox contents of each mock persona.

Phase	Event	Notification	Recipients
2	Requirement created / assigned	`requirement`	Assignee
3	Submitted for review	`review_request`	All reviewing FAs on the project (except the submitter)
3	Requirement approved	`review_approved`	Author + assignee
3	Requirement rejected	`review_rejected`	Author + assignee + milestone PM
3/6	Requirement description revised	`revision_required`	Author, assignee, and assignees of linked tasks/test cases
4	Developer assigned to requirement	`requirement_dev_assigned`	The developer
4/5	Task created / assigned / status changed	`task`	Task assignees
5	QA PIC assigned to an execution file	`execution`	The QA PIC
5	Developer marks Ready for QA	`requirement_ready_for_qa`	Requirement assignee + milestone PM
7	Defect opened	`defect_opened`	QA Lead tier on the project (or requirement author for requirement defects)
7	Defect assigned	`defect_assigned`	The assigned developer
7	Defect status changed	`defect_status_changed`	Reporter + executor of the linked test case
7	Fix ready but test case still failed	`retest_needed`	The QA who last executed the test case
9	UAT pass rate crosses 80%	`uat_milestone_ready`	Milestone PM (once per milestone)
All	Comment posted on a requirement	`comment_posted`	Thread participants

6. Mock-Up Data Preparation Checklist

To stage a complete demo of the flow above, prepare the data set in this order (each item maps to the scenarios named in brackets):

- 1 project (SPARROW) with 11 users assigned across the roles in Section 2.
- 1 primary milestone CR-2026-014 with full phase dates and environment ENV2/ENV4, plus 1 hotfix milestone for contrast (S1.1–S1.4).
- 4 requirements REQ-101…REQ-104 with acceptance criteria; at least two stored AI-analysis runs including one low score (S2.1–S2.4).
- 1 full approval trail (S3.1), 1 blocked self-approval (S3.2), 1 reject-and-revise cycle (S3.3), 1 stale review (S3.4).
- Dev assignments for all approved requirements, including one refused pre-approval assignment and one reassignment (S4.1–S4.4).
- 1 requirement defect raised by a developer and 1 blocked task with a dependency (S5.2, S5.3).
- ~17 test cases (TC-201…TC-217), four of them AI-assisted, all linked to requirements (S6.1–S6.4).
- 1 SIT execution file with 28 rows covering Passed, Failed, Blocked and In Progress results (S7.1–S7.4).

- Defects DEF-0042 (high, closed), DEF-0047 (medium, deferred), DEF-0051 (critical, closed) with full lifecycles (S7.2, S8.2, S8.3).
- 1 UAT execution file with 15 rows, one UAT defect DEF-0058, and the 80% readiness alert (S9.1–S9.4).
- Milestone completion with sign-off, plus 1 production escape DEF-P0004 with escape classification (S10.1–S10.4).
- 7 risks R-01…R-07 across categories and statuses, including one realized; 2 AI risk assessments (high → medium) (S11.1–S11.4).
- 3 lessons-learned entries on the closed milestone (S12.1–S12.3).
- Inbox contents per persona consistent with the notification matrix in Section 5.

End of document. The scenarios are fictional and use placeholder names; align identifiers (REQ / TC / DEF / R numbers) with the seeded database before the presentation.